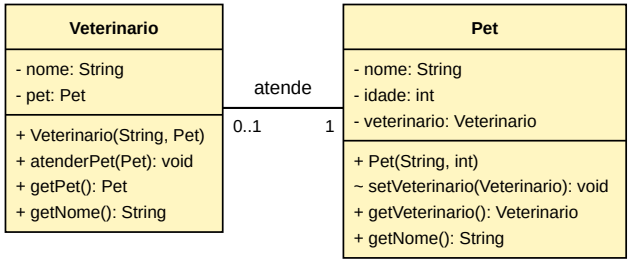
1. Associação 1 : 1 (um para um)

Cada veterinário atende exatamente um pet; cada pet tem no máximo um veterinário.



Regra: multiplicidade máxima 1 de cada lado -> atributo simples nas duas classes.
Os dois lados guardam a referência um do outro (associação bidirecional).
1 no lado do Pet: o construtor exige um Pet e a troca nunca deixa o Veterinario sem pet.
0..1 no lado do Veterinario: o Pet nasce sem veterinário e, quando é trocado, fica livre. Um pet que já tem veterinário é recusado.
setVeterinario sem public (~): só atenderPet altera o vínculo.

linhas que mudam de uma associação para a outra

Veterinario.java + Pet.java

```
public class Veterinario {
    private String nome;
    private Pet pet;                    // exatamente 1 pet

    // nasce com 1 pet: garante o "1" do lado do Pet
    public Veterinario(String nome, Pet pet) {
        this.nome = nome;
        atenderPet(pet);
    }

    public void atenderPet(Pet pet) {
        if (pet.getVeterinario() == this) return; // já é o atual
        if (pet.getVeterinario() != null)         // pet já tem vet
            throw new IllegalStateException("Pet já atendido");
        if (this.pet != null)
            this.pet.setVeterinario(null);        // libera o anterior
        this.pet = pet;
        pet.setVeterinario(this);                 // mantém os dois lados
    }

    public Pet getPet() { return pet; }
    public String getNome() { return nome; }
}

public class Pet {
    private String nome;
    private int idade;
    private Veterinario veterinario;  // 0..1: nenhum ou um vet

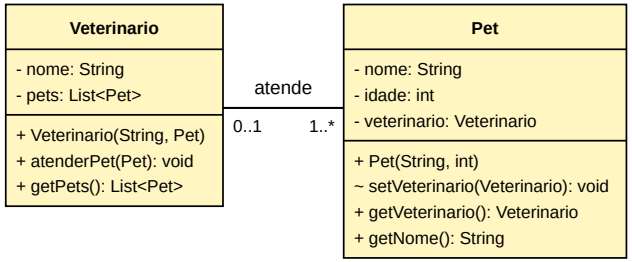
    public Pet(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    // sem public: só Veterinario (mesmo pacote) altera
    void setVeterinario(Veterinario v) {
        this.veterinario = v;
    }

    public Veterinario getVeterinario() { return veterinario; }
    public String getNome() { return nome; }
}
// Uso
Pet rex = new Pet("Rex", 3);         // ainda sem veterinário
Veterinario vet = new Veterinario("Dra. Ana", rex);
vet.atenderPet(new Pet("Mel", 5));   // troca: 'rex' fica livre
```

2. Associação 1 : N (um para muitos)

Um veterinário atende um ou vários pets; cada pet é atendido por no máximo um veterinário.



Regra: o lado com multiplicidade * vira uma coleção (List, Set...).
O lado com multiplicidade máxima 1 continua sendo um atributo simples.
1..* no lado do Pet: o construtor já exige um Pet, e getPets() devolve a lista só para leitura, então ninguém a esvazia por fora.
0..1 no lado do Veterinario: o Pet nasce sem veterinário; atenderPet recusa um Pet que já tem um, e setVeterinario não é public, então o pet nunca fica em duas listas.

linhas que mudam de uma associação para a outra

Veterinario.java + Pet.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Veterinario {
    private String nome;
    private List<Pet> pets = new ArrayList<>();   // muitos pets

    // nasce com 1 pet: garante o mínimo 1
    public Veterinario(String nome, Pet primeiroPet) {
        this.nome = nome;
        atenderPet(primeiroPet);
    }

    public void atenderPet(Pet pet) {
        if (pet.getVeterinario() != null) // só 1 vet por pet
            throw new IllegalStateException("Pet já atendido");
        pets.add(pet);
        pet.setVeterinario(this);
    }

    // somente leitura: de fora não dá para esvaziar
    public List<Pet> getPets() {
        return Collections.unmodifiableList(pets);
    }
}

public class Pet {
    private String nome;
    private int idade;
    private Veterinario veterinario;  // 0..1: nenhum ou um vet

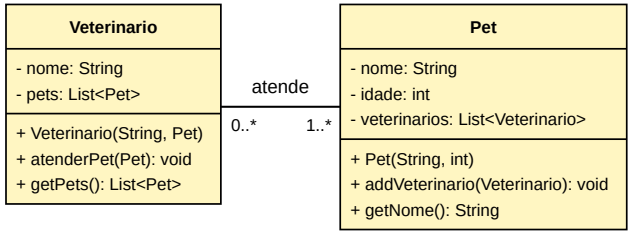
    public Pet(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    // sem public: só Veterinario (mesmo pacote) altera
    void setVeterinario(Veterinario v) {
        this.veterinario = v;
    }

    public Veterinario getVeterinario() { return veterinario; }
    public String getNome() { return nome; }
}
// Uso
Pet mel = new Pet("Mel", 5);         // ainda sem veterinário
Veterinario vet = new Veterinario("Dra. Ana",
                                  new Pet("Rex", 3));
vet.atenderPet(mel);
```

3. Associação N : N (muitos para muitos)

Um veterinário atende um ou vários pets; um pet pode ser atendido por nenhum, um ou vários veterinários.



Regra: * dos dois lados -> coleção nas duas classes.
1..* no lado do Pet: o Veterinario nasce com um Pet, e a ligação de mão dupla dá a esse Pet o seu Veterinario. Se houver remoção, bloqueie quando size() == 1.
0..* no lado do Veterinario: o Pet nasce com a lista vazia.
O teste contains() evita duplicatas e a recursão infinita entre os métodos.
No banco de dados, esse caso vira uma tabela associativa (ex.: atendimento).

linhas que mudam de uma associação para a outra

Veterinario.java + Pet.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Veterinario {
    private String nome;
    private List<Pet> pets = new ArrayList<>();   // muitos pets

    // nasce com 1 pet: garante o mínimo 1
    public Veterinario(String nome, Pet primeiroPet) {
        this.nome = nome;
        atenderPet(primeiroPet);
    }

    public void atenderPet(Pet pet) {
        if (!pets.contains(pet)) {
            pets.add(pet);
            // contains() evita duplicata e laço infinito
            pet.addVeterinario(this);
        }
    }

    public List<Pet> getPets() {
        return Collections.unmodifiableList(pets);
    }
}

public class Pet {
    private String nome;
    private int idade;
    // 0..*: o Pet pode ainda não ter veterinário
    private List<Veterinario> veterinarios = new ArrayList<>();

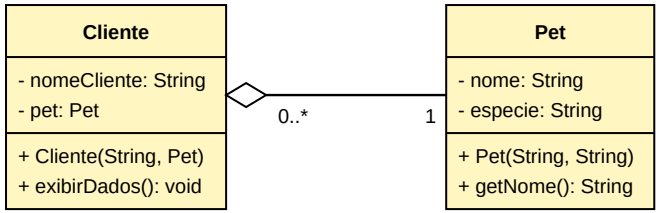
    public Pet(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    public void addVeterinario(Veterinario v) {
        if (!veterinarios.contains(v)) {
            veterinarios.add(v);
            v.atenderPet(this);
        }
    }

    public String getNome() { return nome; }
}
// Uso
Pet rex = new Pet("Rex", 3);          // 0 veterinários
Veterinario ana = new Veterinario("Dra. Ana", rex);
Veterinario bru = new Veterinario("Dr. Bruno", rex);
```

4. Agregação (losango vazado)

O Cliente TEM um Pet, mas o Pet tem vida própria: existe antes e depois do Cliente.



Losango VAZADO no lado do todo (Cliente) = agregação.
Marca no código: o objeto chega PRONTO pelo construtor/setter (new fora).
Ciclo de vida independente: destruir o Cliente não destrói o Pet.
Multiplicidades: 1 no Pet (o construtor exige um Pet); 0..* no Cliente: o Pet pode estar sem dono ou ser compartilhado por vários Clientes, já que nada no código impede.
Para limitar a 0..1, o Pet precisa guardar o dono e o Cliente recusar um Pet que já tem dono (veja o item 6).

linhas que caracterizam a agregação

Cliente.java + Pet.java

```
public class Pet {
    private String nome;
    private String especie;
    public Pet(String nome, String especie) {
        this.nome = nome;
        this.especie = especie;
    }

    public String getNome() { return this.nome; }
}

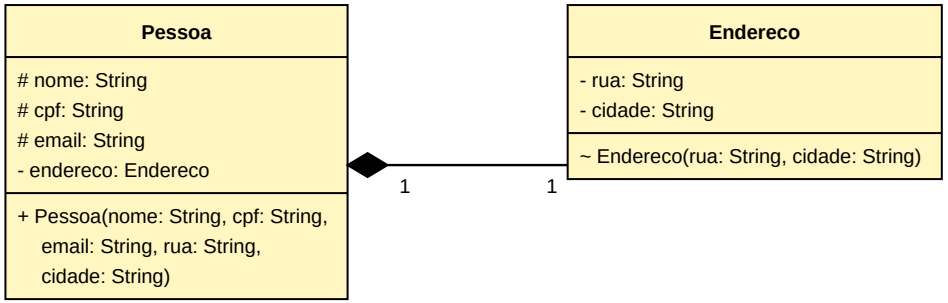
public class Cliente {
    private String nomeCliente;
    // o Pet existe independentemente do Cliente
    private Pet pet;

    // recebe um Pet JÁ CRIADO como parâmetro
    public Cliente(String nome, Pet pet) {
        this.nomeCliente = nome;
        this.pet = pet;
    }

    public void exibirDados() {
        System.out.println("Cliente: " + this.nomeCliente
                           + " | Pet: " + pet.getNome());
    }
}
// Uso
Pet rex = new Pet("Rex", "Cachorro");
Cliente ana = new Cliente("Ana", rex);
Cliente bia = new Cliente("Bia", rex); // mesmo Pet, outro Cliente
ana.exibirDados();
// se 'ana' for descartada, 'rex' continua existindo
```

5. Composição (losango preenchido)

A Pessoa TEM um Endereço que não faz sentido sozinho: nasce e morre com ela.



Losango PREENCHIDO no lado do todo (Pessoa) = composição.
Marca no código: o new acontece DENTRO da classe todo; a parte não vem de fora.
Ciclo de vida dependente: sem Pessoa, aquele Endereço deixa de existir.
O atributo é private e não há getter: a referência ao Endereço nunca sai da Pessoa.
~ (pacote) no construtor de Endereco: só classes do mesmo pacote podem criá-lo.

linhas que caracterizam a composição

Pessoa.java + Endereco.java

```
class Endereco {
    private String rua;
    private String cidade;

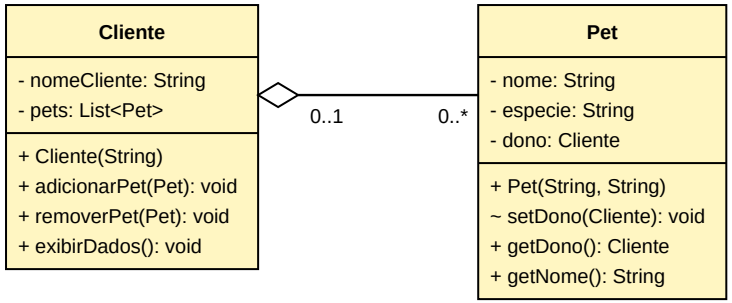
    // construtor sem public: fora do pacote ninguém cria Endereco
    Endereco(String rua, String cidade) {
        this.rua = rua;
        this.cidade = cidade;
    }
}

public class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
    // Composição: Pessoa "tem um" Endereco
    private Endereco endereco;

    public Pessoa(String nome, String cpf, String email,
                  String rua, String cidade) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
        // a Pessoa CRIA o próprio Endereço
        this.endereco = new Endereco(rua, cidade);
    }
}
// Uso
Pessoa p = new Pessoa("Ana", "000.000.000-00",
                      "ana@email.com",
                      "Rua das Flores, 10", "Curitiba");
// não existe referência externa ao Endereco:
// ele nasce e é destruído junto com a Pessoa
```

6. Agregação 0..1 : 0..* (losango vazado)

Um Cliente pode ter nenhum, um ou vários Pets; cada Pet continua existindo por conta própria.



Multiplicidade 0..* -> coleção que pode ficar vazia (nenhum Pet).
0..1 no lado do Cliente: o Pet pode existir sem dono (antes de entrar ou depois de sair da lista).
Para garantir o 0..1, o Pet guarda o dono: adicionarPet recusa um Pet que já tem dono.
Continua sendo agregação: o new do Pet acontece fora e o objeto chega pronto.
remove() tira o Pet da lista, mas não destrói o objeto.

linhas que caracterizam a agregação

Cliente.java + Pet.java

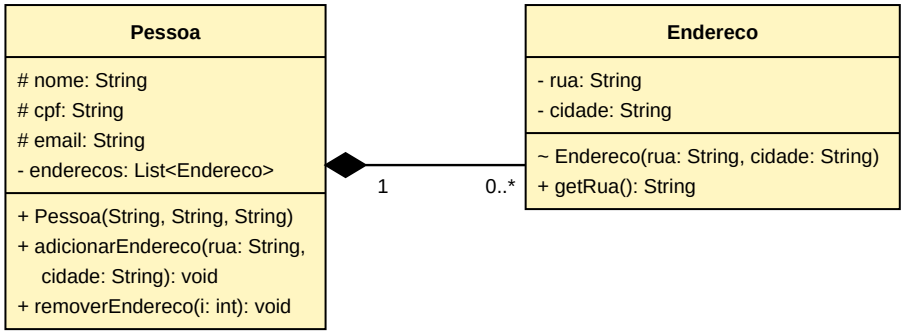
```
import java.util.ArrayList;
import java.util.List;

public class Pet {
    private String nome;
    private String especie;
    private Cliente dono;           // 0..1: nenhum ou um dono
    public Pet(String nome, String especie) {
        this.nome = nome;
        this.especie = especie;
    }
    public String getNome() { return this.nome; }
    public Cliente getDono() { return this.dono; }
    void setDono(Cliente c) { this.dono = c; }  // sem public
}

public class Cliente {
    private String nomeCliente;
    // 0 ou N pets, todos criados FORA do Cliente
    private List<Pet> pets = new ArrayList<>();
    // pode nascer sem nenhum pet -> mínimo 0
    public Cliente(String nome) {
        this.nomeCliente = nome;
    }
    // recebe um Pet JÁ CRIADO
    public void adicionarPet(Pet pet) {
        if (pet.getDono() != null)   // garante o 0..1
            throw new IllegalStateException("Pet já tem dono");
        pets.add(pet);
        pet.setDono(this);
    }
    public void removerPet(Pet pet) {
        if (pets.remove(pet))        // o Pet continua existindo
            pet.setDono(null);       // e fica livre, sem dono
    }
    public void exibirDados() {
        System.out.print("Cliente: " + this.nomeCliente);
        for (Pet p : pets) System.out.print(" | " + p.getNome());
        System.out.println();
    }
}
// Uso
Pet rex = new Pet("Rex", "Cachorro");
Cliente ana = new Cliente("Ana");   // 0 pets
ana.adicionarPet(rex);              // agora 1
ana.removerPet(rex);               // 'rex' sobrevive sozinho
```

7. Composição 1 : 0..* (losango preenchido)

Uma Pessoa pode ter nenhum, um ou vários Endereços, mas todos pertencem só a ela.



Multiplicidade 0..* -> coleção que pode ficar vazia (nenhum Endereço).
Continua sendo composição: o método recebe DADOS e faz o new lá dentro.
Nada de fora guarda referência: remover da lista destrói o Endereço.
~ (pacote) no construtor de Endereco: só classes do mesmo pacote podem criá-lo.

linhas que caracterizam a composição

Pessoa.java + Endereco.java

```
import java.util.ArrayList;
import java.util.List;

class Endereco {
    private String rua;
    private String cidade;
    // construtor sem public: fora do pacote ninguém cria Endereco
    Endereco(String rua, String cidade) {
        this.rua = rua;
        this.cidade = cidade;
    }
    public String getRua() { return this.rua; }
}

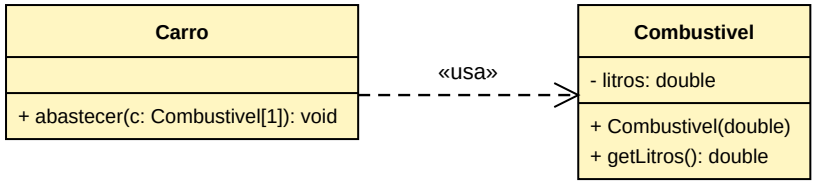
public class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
    // 0 ou N endereços, todos criados DENTRO da Pessoa
    private List<Endereco> enderecos = new ArrayList<>();
    public Pessoa(String nome, String cpf, String email) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
    }

    // recebe DADOS, não objetos: a própria Pessoa dá new
    public void adicionarEndereco(String rua, String cidade) {
        enderecos.add(new Endereco(rua, cidade));
    }

    public void removerEndereco(int i) {
        enderecos.remove(i);    // o Endereco deixa de existir
    }
}
// Uso
Pessoa p = new Pessoa("Ana", "000...", "ana@email.com");
p.adicionarEndereco("Rua das Flores, 10", "Curitiba");
p.adicionarEndereco("Av. Brasil, 500", "Londrina");
// se 'p' deixar de existir, os dois Enderecos somem junto
```

8. Dependência - envio de UM objeto

O Carro usa um Combustivel recebido como parâmetro, sem guardá-lo como atributo.



Seta TRACEJADA com ponta aberta e estereótipo «usa» = dependência.
Não há vínculo estrutural: só uso momentâneo de 1 objeto por chamada.
A referência só existe enquanto o método executa; o objeto continua fora do Carro (relação mais fraca de todas).
Como identificar 1..1: o parâmetro é UM objeto (Combustivel c). No UML, a multiplicidade vai no parâmetro: c: Combustivel[1] (o [1] pode ser omitido, é o padrão).

linhas que mostram a ausência de vínculo estrutural
parâmetro que cria a dependência

Carro.java + Combustivel.java

```
public class Combustivel {
    private double litros;

    public Combustivel(double litros) {
        this.litros = litros;
    }

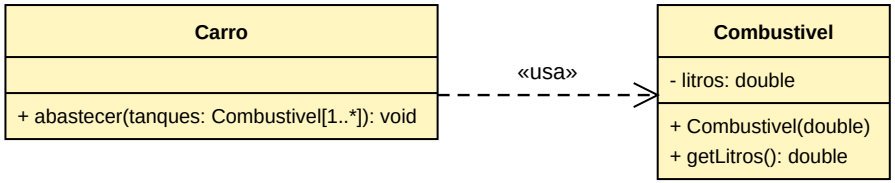
    public double getLitros() { return this.litros; }
}

public class Carro {

    // nenhum atributo do tipo Combustivel:
    // a classe apenas USA o objeto durante o método
    public void abastecer(Combustivel c) {
        System.out.println(c.getLitros());
    }
}
// Uso
Carro carro = new Carro();
carro.abastecer(new Combustivel(45.0));
// terminado o método, 'c' sai de escopo e o Carro
// não guarda nenhuma referência ao Combustivel
```

9. Dependência - envio de N objetos

O Carro usa VÁRIOS Combustíveis recebidos numa lista, sem guardá-los como atributo.



Como identificar 1..N: o parâmetro é uma COLEÇÃO (List<Combustivel>). No UML, a multiplicidade vai no parâmetro: tanques: Combustivel[1..*].
A seta de dependência não leva multiplicidade nas pontas; quem diz "um" ou "vários" é o parâmetro.
Mínimo 1 (1..*): o método recusa lista vazia. Se aceitasse vazia, seria [0..*].
Continua sendo dependência: seta tracejada «usa», a lista chega por parâmetro.
Regra prática: se a lista virasse atributo, seria associação (aí sim com multiplicidade).

linhas que mudam em relação ao item 8
parâmetro que cria a dependência

Carro.java + Combustivel.java

```
import java.util.List;

public class Combustivel {
    private double litros;

    public Combustivel(double litros) {
        this.litros = litros;
    }

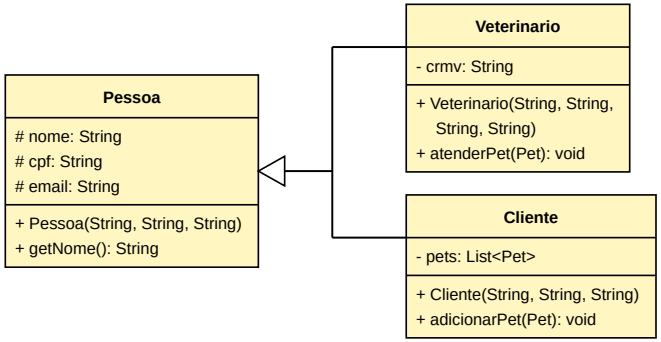
    public double getLitros() { return this.litros; }
}

public class Carro {

    // continua sem atributo: muda só a forma do parâmetro
    public void abastecer(List<Combustivel> tanques) {
        if (tanques.isEmpty())          // garante o mínimo 1
            throw new IllegalArgumentException("Informe ao menos 1");
        for (Combustivel c : tanques) {
            System.out.println(c.getLitros());
        }
    }
}
// Uso
Carro carro = new Carro();
carro.abastecer(List.of(new Combustivel(45.0),
                        new Combustivel(12.5)));
// a lista e os objetos continuam fora do Carro:
// 1 Carro usa N Combustíveis apenas durante o método
```

10. Herança - extends (é-um)

Veterinario e Cliente SÃO Pessoa: herdam atributos e métodos e acrescentam o que é seu.



Triângulo VAZADO em linha cheia, apontando para a superclasse = generalização.
Uma superclasse pode ter quantas subclasses forem necessárias (aqui, duas).
Sem multiplicidade: não é um objeto dentro de outro, e sim um tipo derivado de outro.
'protected' (#) libera o atributo para as subclasses; super(...) chama o construtor pai.
Construtores não são herdados: cada subclasse declara o seu e chama super(...).

linhas que caracterizam a herança

palavra-chave que estabelece o vínculo

Pessoa.java + Veterinario.java + Cliente.java

```
import java.util.ArrayList;
import java.util.List;
// Pet: a classe do item 4 (omitida no diagrama: foco na herança)

public class Pessoa {                  // superclasse
    protected String nome;             // visível nas subclasses
    protected String cpf;
    protected String email;

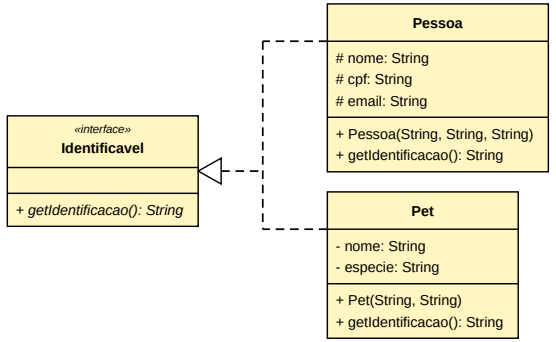
    public Pessoa(String nome, String cpf, String email) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
    }
    public String getNome() { return this.nome; }
}

public class Veterinario extends Pessoa {
    private String crmv;               // atributo só da subclasse
    public Veterinario(String nome, String cpf,
                       String email, String crmv) {
        super(nome, cpf, email);       // reaproveita a superclasse
        this.crmv = crmv;
    }
    public void atenderPet(Pet pet) {
        // 'nome' vem herdado de Pessoa
        System.out.println(nome + " atende " + pet.getNome());
    }
}

public class Cliente extends Pessoa {
    private List<Pet> pets = new ArrayList<>();
    public Cliente(String nome, String cpf, String email) {
        super(nome, cpf, email);       // mesma superclasse
    }
    public void adicionarPet(Pet pet) { pets.add(pet); }
}
// Uso: polimorfismo - os dois SÃO Pessoa
Pessoa[] pessoas = { new Veterinario("Ana", "000...",
                         "ana@email.com", "PR-1234"),
                     new Cliente("Bruno", "111...", "b@email.com") };
for (Pessoa p : pessoas)
    System.out.println(p.getNome());   // método herdado
```

11. Interface - implements (sabe-fazer)

Pessoa e Pet não têm parentesco, mas ambas assinam o mesmo contrato Identificavel.



Triângulo VAZADO em linha TRACEJADA apontando para a interface = realização.
A interface declara métodos abstratos (só a assinatura, em *itálico* no UML); cada classe escreve sua própria implementação. Desde o Java 8 ela também aceita métodos default/static e constantes.
Extends: herda comportamento (é-um). Implements: assume um contrato (sabe-fazer).
Uma classe estende UMA superclasse, mas pode implementar VÁRIAS interfaces.

linhas que caracterizam o contrato

palavra-chave que estabelece o vínculo

Identificavel.java + Pessoa.java + Pet.java

```
public interface Identificavel {
    // método abstrato: só a assinatura, sem corpo
    String getIdentificacao();
}

public class Pessoa implements Identificavel {
    protected String nome;
    protected String cpf;
    protected String email;
    public Pessoa(String nome, String cpf, String email) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
    }
    @Override
    public String getIdentificacao() {
        return "CPF " + this.cpf;
    }
}

public class Pet implements Identificavel {
    private String nome;
    private String especie;
    public Pet(String nome, String especie) {
        this.nome = nome;
        this.especie = especie;
    }
    @Override
    public String getIdentificacao() {
        return this.nome + " (" + this.especie + ")";
    }
}
// Uso: mesmo contrato para tipos sem parentesco entre si
// (o import fica no arquivo do uso: import java.util.List;)
List<Identificavel> lista = List.of(
    new Pessoa("Ana", "000...", "ana@email.com"),
    new Pet("Rex", "Cão"));
for (Identificavel i : lista)
    System.out.println(i.getIdentificacao());
```